



Image analysis (MicroICS) hands-on session: exercises

Part of: "Deep dive into the statistical approaches used to validate potential standardised laboratory methods"

In this session we will use the tool MicroICS to analyse 3D images of biofilms. This tool consists of three interoperable modules: 1) Feature calculation – extracting structural features from 3D confocal fluorescence images; 2) Model training – training machine learning models on those features with respect to your biological question; and 3) Inference – applying the trained model to unknown images to predict the property of the strain in the image.

The golden thread running through the image analysis tool MicroICS follows the pattern below, and it applies to your own biological question just as much as to questions studied on the Training School (TS):

Biological question → planning and image acquisition → linking image and biological question through labels → calculation of structural features → model training → **answer to what separates the groups** → prediction on unknown (treated) images → **answer to whether a treatment changes the structure**.

The biological question for this TS hands-on will be: can we distinguish between the eight *Listeria* strains solely on the basis of their 3D structure, and what causes the differences? Most of the steps carried out will be used to demonstrate how to use the software, and the results are illustrative; to obtain reliable answers to this question you will also need to complete the homework, where you will analyse the whole dataset.

The main takeaway from this session is that you can formulate a biological question that fits your area of interest and plan an analytical approach using the toolset presented.

Objective 1: Turn a biological question into a label

The label is the ground truth that you attach to each image, written into its filename (image) after `_st_`. Change the label and you change the question; the pipeline never changes.

Do: open the training image folder and look at the filenames. Find the `_st_` part. That is the label.

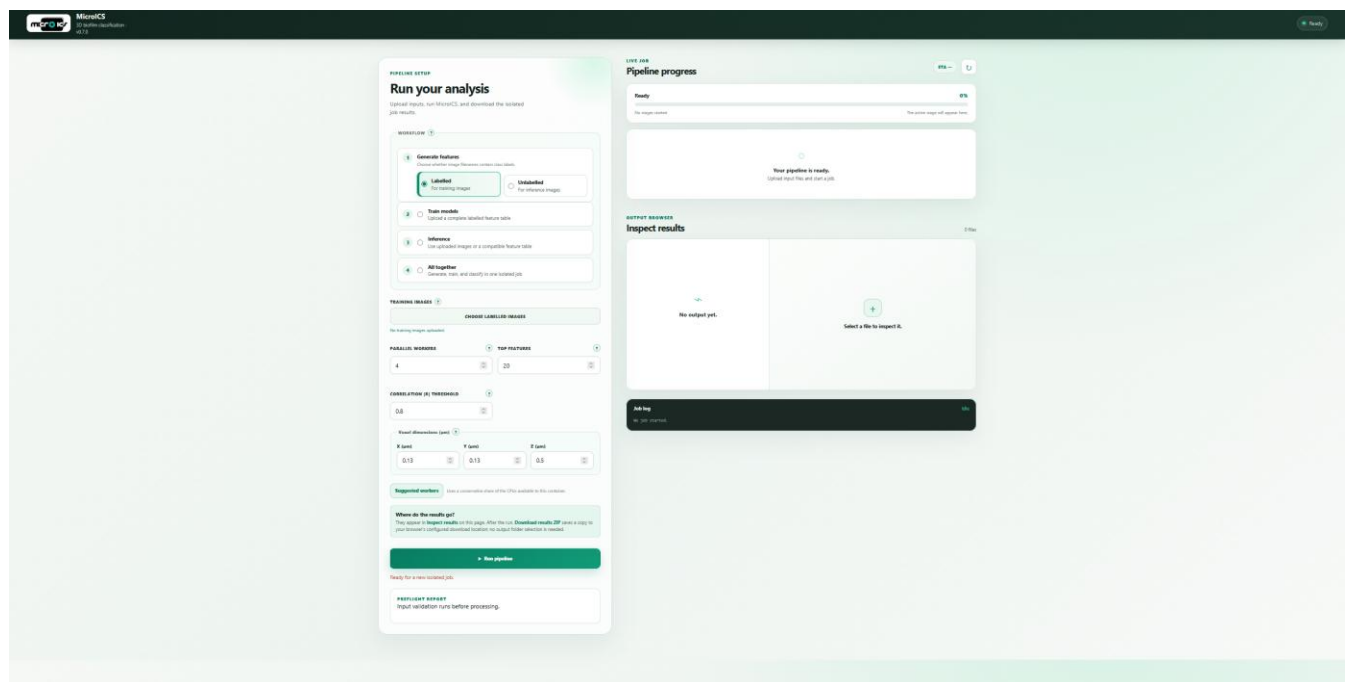
Think and write: How would you name the images to distinguish clinically relevant from non-relevant strains? How would you label images for your question?

Take home: your question becomes a machine learning task the moment you can write its answer into the filename.

Objectives 3 – 5: Install and run MicroICS independently

Installation

- Open the document “Installation Guide” and start MicroICS (**Restarting the existing container of MicroICS**). The window that opens should look like this:



Generate features

To run “generate features”, you will need sample images, which you can download here:

<https://portal.ijs.si/nextcloud/s/LC93n9A97Ekme7X>.

Do:

1. Choose “Generate features” in mode “Labelled”. Labelled mode extracts the label from each filename, so that the model can link the features to the label during training.
2. Point the software to the folder containing your images in TIFF format, using “Choose labelled images”.
1. Set the running options. “Parallel workers” – how much of your CPU to dedicate to the analysis: use full capacity when you are not otherwise using the computer, and less if you are working on it at the same time. Suggested workers will provide optimal settings for running. At this point, “Top features” and “Correlation |R| threshold” do not matter.
3. Adjust the voxel dimensions (the pixel size in micrometres) to match your images.
4. Click “Run pipeline”. When the run starts, the software also checks the image naming and reports the number of classes it found (“Preflight report”).

The screenshot shows the 'Generate features' web interface. On the left, a sidebar contains four main sections: '1 Generate features' (highlighted with a box and callout 1), '2 Train models', '3 Inference', and '4 All together'. The 'Generate features' section has two radio buttons: 'Labelled' (selected, with callout 1) and 'Unlabelled'. Below this are three other radio buttons: 'Train models', 'Inference', and 'All together'. The main content area on the right is divided into several sections. At the top is 'TRAINING IMAGES' (callout 2) with a 'CHOOSE LABELLED IMAGES' button and a message 'No training images uploaded.'. Below this is a section with 'PARALLEL WORKERS' (set to 4, callout 3) and 'TOP FEATURES' (set to 20, callout 3). Below that is 'CORRELATION |R| THRESHOLD' (set to 0.8, callout 3). Then is 'Voxel dimensions (µm)' (callout 4) with three input fields: 'X (µm)' (0.13), 'Y (µm)' (0.13), and 'Z (µm)' (0.5). Below this is a 'Suggested workers' section with a note. Then is a 'Where do the results go?' section with a note. At the bottom is a large green 'Run pipeline' button (callout 5) and a 'PREFLIGHT REPORT' section with the text 'Input validation runs before processing.'

See the results: View the results by downloading the ZIP file. Open datafile.tsv – it has one row per image and roughly 2,700 columns, each representing a structural measurement (shape, texture, intensity, or spatial arrangement).

Take home: The model requires features as input, not images. Thus, the first step is to calculate the features and output a data file that the model uses for training. Details of the output files are described in the homework document.

Train a model

Training the model to output meaningful results takes too long, so we provide a pre-made model for the training school and do not spend the session waiting. **Thus, skip this part for now, but not for the homework. The pre-made models are available here: <https://portal.ijs.si/nextcloud/f/53497958>**

Do:

2. Choose “Train models.”
3. Navigate to the folder containing calculated features (datafile.tsv) using “Choose feature table”.
4. Set the running options: “Parallel workers” – how much of your CPU to dedicate to the analysis: full capacity when you are not otherwise using the computer, and less if you are working on it at the same time. Suggested workers will provide optimal settings for running. “Correlation threshold” – choose how many of the most important features to analyse, and the threshold for the analysis.
“Unit of replication” – specify what counts as an independent experiment (day, plate, well, or position within the well). This is important for testing the trained model correctly.
5. Choose which learner to train on your data: a single learner and choose learner (model) from dropdown menu, or all those currently available.
6. Click “Run pipeline.” When the run starts, the software also checks the input datafile for missing values (NaN, empty cells, etc.).

The screenshot shows the 'Train a model' interface with five numbered callouts:

- 1** Points to the 'Train models' option in the left sidebar, which is selected. Below it are 'Inference' and 'All together' options.
- 2** Points to the 'COMPLETE FEATURE TABLE' section, which includes a 'CHOOSE FEATURE TABLE' button and a message 'No feature table uploaded.'
- 3** Points to the 'PARALLEL WORKERS' and 'TOP FEATURES' sections. 'PARALLEL WORKERS' is set to 4, and 'TOP FEATURES' is set to 20. Below these are 'CORRELATION |R| THRESHOLD' (set to 0.8) and 'UNIT OF REPLICATION' (set to 'Well').
- 4** Points to the 'Learners' section, which has two options: 'One learner' (selected, with subtext 'Focused, faster training') and 'All learners' (with subtext 'Benchmark every algorithm'). Below this is a 'LEARNER' dropdown menu set to 'Random Forest'.
- 5** Points to the 'Run pipeline' button at the bottom of the form.

Additional text in the interface includes: 'Suggested workers' (Uses a conservative share of the CPUs available to this container.), 'Where do the results go?' (They appear in **Inspect results** on this page. After the run, **Download results ZIP** saves a copy to your browser's configured download location; no output folder selection is needed.), and 'PREFLIGHT REPORT' (Input validation runs before processing.).

For now, focus on these outputs and set the rest aside:

- **classification_all** – performance of each classifier, check if it beats the majority-class baseline.
- **ablation_ranking_all** – the smallest number of features needed to maintain optimal accuracy.
- **rankings_label** – the structural features that best distinguish the strains. Here you can find information on which structural properties differ between the strains.
- **rankings_date** – compared with rankings_label, it identifies features that also predict the imaging date; any feature ranking highly in both is tracking acquisition as much as biology and should therefore be interpreted with caution.

Take home: The model outputs two types of information: how well it performs on a given dataset for the specified biological question, and a ranked list of features that indicate what differs between groups and flag where those differences may reflect technical artefacts rather than biology.

The remaining output files are described in the homework document and are not required to understand the framework.

Inference

Inference consists of two steps: first, calculate features from the new images to obtain a list of features that the model can then use to make an inference on them, which is done in the second step.

Do step (1) — generate features from the new images:

1. Choose “Generate features” in “Unlabelled” mode.
2. Point the software to the folder with the images for inference, using “Choose inference images”.
3. Set “Parallel workers” – how much of your CPU to dedicate to the analysis: full capacity when you are not otherwise using the computer, less if you are working on it at the same time. Suggested workers will provide optimal settings for running.
“Top features” and “Correlation |R| threshold” do not matter here. Adjust the voxel dimensions (the pixel size in micrometres) to match your images.
4. Click “Run pipeline”. When the run starts, the software checks the image naming but **does not read labels** – in unlabelled mode the labels are ignored because these images are the unknowns to be classified.

1 **Generate features**
Choose whether image filenames contain class labels.

☐ **Labelled**
For training images

☒ **Unlabelled**
For inference images

2 **Train models**
Upload a complete labelled feature table

3 **Inference**
Use uploaded images or a compatible feature table

4 **All together**
Generate, train, and classify in one isolated job

2 **IMAGES TO CLASSIFY** ?
CHOOSE INFERENCE IMAGES
No inference images uploaded.

3 **PARALLEL WORKERS** ? **TOP FEATURES** ?
4 20

CORRELATION |R| THRESHOLD ?
0.8

4 **Voxel dimensions (µm)** ?
X (µm) 0.13 **Y (µm)** 0.13 **Z (µm)** 0.5

Suggested workers Uses a conservative share of the CPUs available to this container.

Where do the results go?
They appear in **Inspect results** on this page. After the run, **Download results ZIP** saves a copy to your browser's configured download location; no output folder selection is needed.

5 **Run pipeline**
Ready for a new isolated job.

Do step (2) — run the model on those features:

1. Choose “Inference”.
2. Point the software to the folder containing inference images using “Choose inference images” **OR** point the software to the feature table (the features you just generated in unlabelled mode) using “Choose feature table”.
3. Point the software to the folder containing the trained model using the drop-down menu “Trained models”. All trained models are saved in the Docker folder, so use the drop-down menu to choose the appropriate model. Models are saved by the date they were trained, followed by the number of models trained (one when only one model was selected, or six when all learners were selected), and then a code created by Docker for tracking in case of problems. If you would like to delete an old model, or a model you ran as a test, select the model from the drop-down menu and click “Delete selected job”. You can also import trained models using “Import _joblib files”. This takes you to the folder with models; select the models you would like to import (all or just one). Once imported, they will be used for inference.
4. Set “Parallel workers” as before. Suggested workers will provide optimal settings for running.
5. Click “Run pipeline”.

The screenshot shows a web-based interface for a machine learning pipeline. On the left, a sidebar contains four main steps: 1. Generate features, 2. Train models, 3. Inference (highlighted with a green border and a callout '1'), and 4. All together. The main panel on the right is divided into several sections. The top section, 'IMAGES TO CLASSIFY', has a 'CHOOSE INFERENCE IMAGES' button (callout '2') and a status message 'No inference images uploaded.'. Below it, the 'COMPLETE FEATURE TABLE' section has a 'CHOOSE FEATURE TABLE' button (callout '2') and a status message 'No feature table uploaded.'. The 'TRAINED MODELS' section features a dropdown menu 'Select a training job' (callout '3'), a description of the options, and buttons for 'IMPORT _JOBLIB FILES' and 'Delete selected job'. The 'PARALLEL WORKERS' section has a dropdown set to '4' (callout '4'). The 'TOP FEATURES' section has a dropdown set to '10'. The 'CORRELATION |R| THRESHOLD' section has a dropdown set to '0.8'. Below these are 'Voxel dimensions (µm)' for X (0.13), Y (0.13), and Z (0.5). A 'Suggested workers' section notes 'Uses a conservative share of the CPUs available to this container.'. A 'Where do the results go?' section explains that results appear in 'Inspect results' and can be downloaded as a ZIP. At the bottom, a large green 'Run pipeline' button (callout '5') is shown with the status 'Ready for a new isolated job.'

1 Generate features
Choose whether image filenames contain class labels.

☐ Labeled
For training images

☐ Unlabelled
For inference images

2 Train models
Upload a complete labelled feature table

3 Inference
Use uploaded images or a compatible feature table

4 All together
Generate, train, and classify in one isolated job

2 IMAGES TO CLASSIFY ?
CHOOSE INFERENCE IMAGES
No inference images uploaded.

2 COMPLETE FEATURE TABLE ?
CHOOSE FEATURE TABLE
No feature table uploaded.

3 TRAINED MODELS ?
Select a training job
Options show when the job was trained and which learner(s) it used. Import a compatible .joblib model below, or delete an old selected job. Persist /data with a Docker volume to keep jobs across container replacement.
IMPORT _JOBLIB FILES Delete selected job
Select the model and matching metadata ".joblib" files together when available.

4 PARALLEL WORKERS ? TOP FEATURES ?
4 10
CORRELATION |R| THRESHOLD ?
0.8

Voxel dimensions (µm) ?
X (µm) Y (µm) Z (µm)
0.13 0.13 0.5

Suggested workers Uses a conservative share of the CPUs available to this container.

Where do the results go?
They appear in **Inspect results** on this page. After the run, **Download results ZIP** saves a copy to your browser's configured download location; no output folder selection is needed.

5 Run pipeline
Ready for a new isolated job.

Read the results, in the following order:

1. **inference_summary** — Check the run analysis, e.g. detect incorrectly read labels during training, etc.
2. **rf_predictions** — the predicted class for each image.
3. **rf_probabilities** — the confidence behind each prediction.
4. **Explanations** — a collection of SHAP-based analyses (tables and plots) showing which features drove the model's predictions and how.

Take home: Inference shows how the model interprets unknown images: it provides a prediction and an associated probability indicating how confident the model is in that decision. Low confidence means the image does not clearly belong to any class the model was trained on – it may resemble a phenotype the model has not seen before. Inference also returns the features that drove the prediction.

Now it's your turn

- Have fun tonight and let diffuse thinking do the work.
- Tomorrow, read back over what we covered today and highlight anything you did not follow.
- Check the theoretical part to see whether it answers those questions.
- Repeat this exercise.
- Read the paper: <https://www.nature.com/articles/s41522-026-01083-8>.
- Then move on to the homework, where you run the full dataset and uncover much more detail.